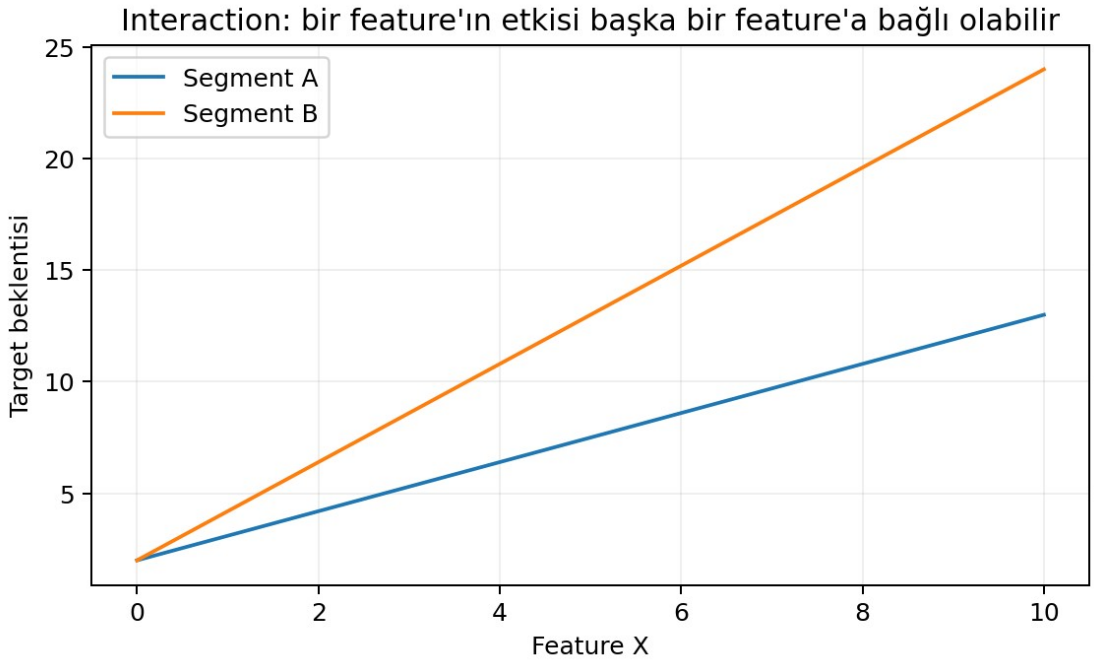
19. FEATURE ENGINEERING

Dönüşümler • Encoding • Scaling • Interaction • Time Features • Leakage



Bu modülün amacı
Feature engineering'i 'daha fazla sütun üretmek' olarak değil; ham veriyi modelin öğrenebileceği, zaman açısından geçerli ve iş anlamı taşıyan sinyallere dönüştürme süreci olarak öğrenmek.

Kazanım	Modül sonunda
Feature düşüncesi	Ham kolon, derived feature, target ve leakage arasındaki farkı netleştirebilirsin.
Sayısal dönüşümler	Log, binning, scaling ve robust dönüşümleri doğru yerde kullanabilirsin.
Categorical encoding	One-hot, ordinal, frequency ve target encoding arasındaki trade-off'ları bilirsin.
Interaction	Feature'lar arası sinerjiyi model tipine göre tasarlayabilirsin.
Time features	Lag, rolling, recency ve seasonal feature'ları future leakage yaratmadan üretebilirsin.
Pipeline	Feature engineering'i reproducible ve leakage-safe pipeline içine alabilirsin.

1. Feature engineering nedir?

Feature engineering, ham veriden modelin tahmin problemi için daha kullanışlı temsiller üretme sürecidir. İyi feature modelin sinyali görmesini kolaylaştırır; kötü feature noise, leakage veya gereksiz complexity üretir.

Ham veri	Feature örneği
order_date	days_since_last_order, order_month, is_weekend
birth_date	age_at_prediction_time
transactions	last_30d_spend, avg_basket_90d, txn_count_7d
text review	TF-IDF, sentiment score, embedding
latitude/longitude	distance_to_store, region
price + quantity	basket_value = price * quantity

Ana ilke
Feature yalnızca tahmin anında erişilebilen bilgiden üretilmelidir. Aksi halde data leakage oluşur.

2. Feature engineering neden modelden önce gelir?

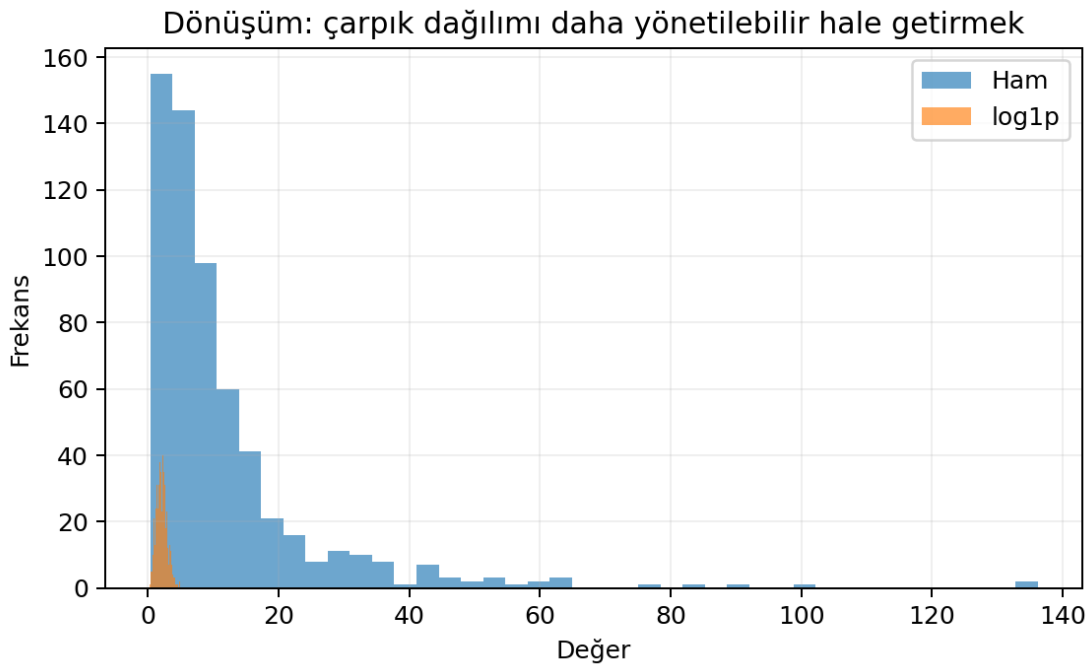
- Model yalnızca verilen representation üzerinden öğrenir.
- Domain knowledge ham veride görünmeyen sinyali açığa çıkarabilir.
- Bazı modeller feature scaling/encoding'e çok duyarlıdır.
- İyi feature daha basit modeli güçlü hale getirebilir.
- Production'da feature üretimi train-serving consistency gerektirir.

Pratik gerçek
Tabular ML'de çoğu zaman küçük model farklarından daha büyük kazanç, doğru feature ve leakage-safe validation'dan gelir.

3. Feature türleri

Tür	Örnek	Not
Numeric continuous	income, price	Scale / transform düşünülebilir.
Numeric discrete	order_count	Count distribution olabilir.
Nominal categorical	city, device	Sıra yok.
Ordinal categorical	low/medium/high	Doğal sıra var.
Datetime	event_time	Seasonality/recency türetilebilir.
Text	review_text	Sparse veya dense representation.
Geo	lat/lon	Distance/region feature.
Relational	customer-order	Aggregate/window feature.

4. Sayısal dönüşümler



Dönüşüm	Ne zaman?
log1p(x)	Pozitif ve sağ çarpık count/money feature'lar.
sqrt(x)	Orta düzey sağ çarpıklık.
clip / winsorize	Aşırı tail değerlerin etkisini sınırlamak gerektiğinde.
rank / quantile transform	Dağılımı daha uniform/normal yapmak gerektiğinde.
power transform	Box-Cox / Yeo-Johnson gibi dağılım dönüşümleri.

Model bağımlılığı
Tree-based modeller çoğu monotonik dönüşüme daha az duyarlıdır; linear, distance-based ve neural modeller scale/distribution'dan daha fazla etkilenebilir.

5. Scaling ve normalization

Yöntem	Formül fikri	Kullanım
StandardScaler	(x-mean)/std	Linear, SVM, KNN, PCA.
MinMaxScaler	[0,1] aralığı	Bounded input gereken durumlar.
RobustScaler	median/IQR	Outlier'a daha dayanıklı scale.
L2 normalization	vektör normunu 1 yap	Text/embedding cosine tabanlı işlemler.

Leakage
Scaler yalnızca training fold üzerinde fit edilmelidir. Tüm dataset ortalama/std değerini kullanmak validation bilgisini train'e sızdırır.

6. Binning / discretization

Continuous feature'ı aralıklara bölmek linear olmayan ilişkileri basit modelde görünür hale getirebilir.

```
age_band = pd.cut(
    df["age"],
    bins=[0, 25, 35, 50, 65, 120],
    labels=["18-25", "26-35", "36-50", "51-65", "65+"]
)
```

Risk
Keyfi bin sınırları bilgi kaybı yaratabilir. Binning'i domain veya validation gerekçesiyle kullan.

7. One-Hot Encoding

Nominal kategoriler için her kategoriye binary sütun üretir. Linear modelde güçlü default yaklaşımdır.

```
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(
    handle_unknown="ignore"
)
```

Avantaj	Risk
Basit ve güvenli	High-cardinality feature'da feature explosion.
Kategori sırası varsaymaz	Sparse matrix büyüyebilir.
Pipeline ile kolay	Unseen category yönetimi gerekir.

8. Ordinal Encoding

Kategorilerin doğal sırası varsa sayısal sıra kodlanabilir.

```
mapping = {
    "low": 0,
    "medium": 1,
    "high": 2
}
df["risk_level_ord"] = df["risk_level"].map(mapping)
```

Dikkat
Nominal kategoriyi 0/1/2 diye kodlamak modele sahte uzaklık ve sıra bilgisi verebilir.

9. Frequency / Count Encoding

Kategoriye görülme sıklığı veya count atanır. High-cardinality kategorilerde compact representation olabilir.

```
freq = train["merchant_id"].value_counts(normalize=True)
train["merchant_freq"] = train["merchant_id"].map(freq)
valid["merchant_freq"] = valid["merchant_id"].map(freq).fillna(0)
```

Training-only
Frequency map de yalnızca training verisinden çıkarılmalıdır.

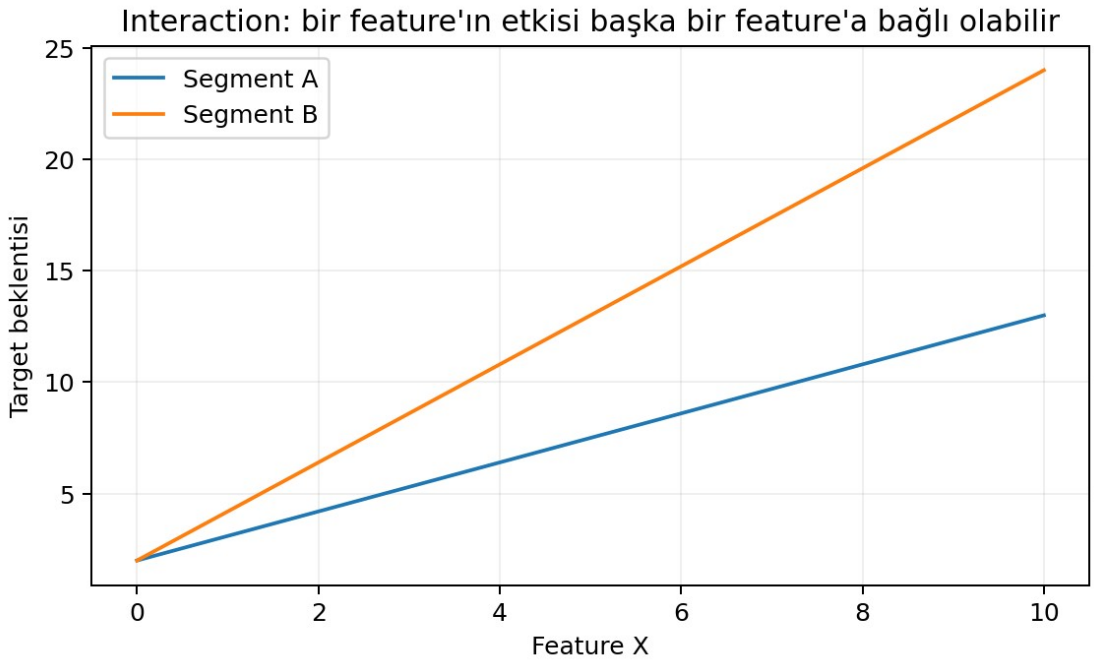
10. Target Encoding

Kategori için target ortalaması gibi supervised özet kullanır. Güçlüdür ama leakage riski yüksektir.

Yanlış	Doğru
Tüm dataset category mean target	Out-of-fold / fold-aware target encoding
Validation target'ını encoding'e katmak	Sadece train fold target'ı ile map oluşturmak
Rare category'yi ham mean ile bırakmak	Smoothing / global prior kullanmak

Kritik
Target encoding'i pipeline/CV farkındalığı olmadan uygulamak sahte validation performansı üretmenin en kolay yollarından biridir.

11. Interaction features



Bir feature'ın target üzerindeki etkisi başka bir feature seviyesine bağlıysa interaction vardır.

```
df["price_per_unit"] = df["revenue"] / df["quantity"].clip(lower=1)
df["age_income_interaction"] = df["age"] * df["income"]
```

Model türü
Linear modeller interaction'ı explicit feature ister. Tree/boosting modeller interaction'ları doğal olarak keşfedebilir; yine de domain-driven interaction fayda sağlayabilir.

12. Polynomial features

x^2, x^3 ve cross-products linear modelin nonlinear ilişki yakalamasına izin verir.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(
    degree=2,
    include_bias=False
)
```

Risk
Feature sayısı kombinatoryal artar. Regularization ve CV olmadan overfitting kolaydır.

13. Ratio ve domain features

Ham kolonlar	Derived feature	İş anlamı
revenue, orders	avg_order_value	Sipariş başına harcama
debt, income	debt_to_income	Borç yükü
clicks, impressions	ctr	Etkileşim oranı
returns, orders	return_rate	Kalite / memnuniyet sinyali
distance, delivery_time	speed_proxy	Lojistik davranışı

Domain > rastgele kombinasyon
Her iki sütunu çarpmak feature engineering değildir. Feature'ın iş mekanizmasını temsil etmesi tercih edilir.

14. Count, recency, frequency, monetary (RFM)

Müşteri davranışında temel feature ailesidir.

Feature	Tanım
Recency	Son işlemden prediction time'a geçen süre
Frequency	Belirli lookback window içindeki işlem sayısı
Monetary	Lookback window harcama toplamı/ortalaması
Tenure	İlk gözlemden prediction time'a süre

Point-in-time correctness
RFM feature'ları prediction tarihinden sonraki işlemleri asla içermemelidir.

15. Aggregate features

Relational veriyi entity seviyesine özetlemek tabular ML'in en güçlü tekniklerinden biridir.

```
customer_agg = (
    orders.groupby("customer_id")
    .agg(
        orders_90d=("order_id", "nunique"),
        spend_90d=("amount", "sum"),
        avg_basket_90d=("amount", "mean")
    )
)
```

Grain kontrolü
Feature tablosunun grain'i target grain ile uyuşmalı. Join sonrası satır çoğalması feature ve target'ı bozabilir.

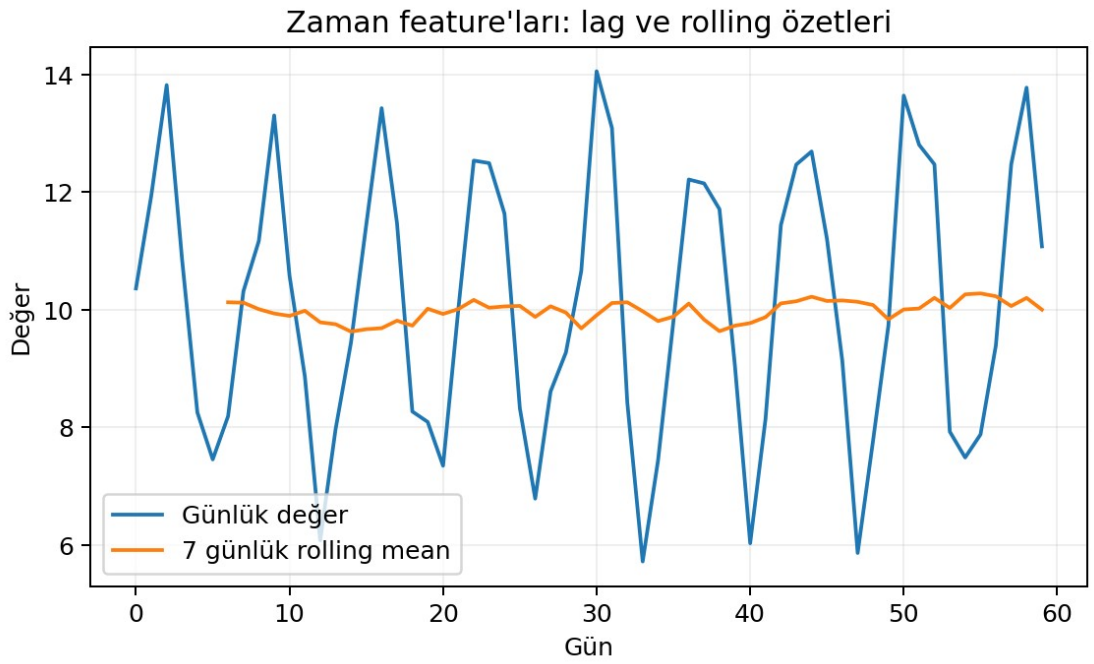
16. Datetime feature engineering

Feature	Örnek
Calendar	hour, day_of_week, month, quarter
Flag	is_weekend, is_month_end, is_holiday
Recency	days_since_last_event
Tenure	days_since_signup
Cyclical	sin/cos(hour), sin/cos(day_of_week)

Cyclical encoding
Saat 23 ile 0 sayısal olarak uzak ama zaman döngüsünde komşudur. Sin/cos encoding bu yapıyı temsil eder.

```
df["hour_sin"] = np.sin(2*np.pi*df["hour"]/24)
df["hour_cos"] = np.cos(2*np.pi*df["hour"]/24)
```

17. Lag features



Lag feature geçmiş target/feature değerlerini bugünkü tahmin için kullanır.

```
df["sales_lag_1"] = df["sales"].shift(1)
df["sales_lag_7"] = df["sales"].shift(7)
```

Future leakage
shift(-1) gibi gelecek değerleri kullanan feature prediction probleminde leak'tir.

18. Rolling window features

Son N gözlem/gün içindeki özetler trend ve momentum sinyali sağlar.

```
df["sales_ma_7"] = (
    df["sales"]
      .shift(1)
      .rolling(7)
      .mean()
)
```

Neden shift(1)?
Bugünkü target'ı bugünkü rolling feature içine almamak için. Forecasting'te rolling'i genellikle geçmişe kaydırmak gerekir.

19. Window tasarımı

Window	Yakalanan sinyal
1-7 gün	Kısa dönem momentum
30 gün	Aylık davranış
90 gün	Daha stabil customer behavior
365 gün	Uzun dönem seasonality / lifetime

Multi-window features
Aynı sinyal için 7d, 30d, 90d feature'ları modelin kısa ve uzun dönem davranışı ayırmasını sağlar.

20. Trend ve change features

```
df["spend_change_7_30"] = (
    df["spend_7d"] / df["spend_30d"].clip(lower=1e-6)
)

df["activity_delta"] = (
    df["events_7d"] - df["events_prev_7d"]
)
```

Churn örneği
Mutlak kullanım seviyesi kadar kullanımın son haftalarda düşmesi güçlü churn sinyali olabilir.

21. Missingness as feature

Eksik değer bazen veri kalitesinden değil, davranıştan kaynaklanır. Missing indicator ayrı sinyal taşıyabilir.

```
df["income_missing"] = df["income"].isna().astype(int)
```

Dikkat
Missing indicator kullanmak imputation ihtiyacını ortadan kaldırmaz; model ve pipeline'a göre ikisi birlikte kullanılabilir.

22. Outlier-resistant features

- Clip/winsorize yalnız domain gerekçesi varsa.
- Median/IQR tabanlı robust scale.
- log1p ile heavy-tail count/money.
- Percentile/rank feature.

Outlier silmek ≠ feature engineering
Gerçek yüksek değerleri veri hatası sanıp silmek sinyali yok edebilir.

23. Text feature engineering

Yaklaşım	Ne üretir?
Length features	Karakter/kelime sayısı
Keyword flags	Domain anahtar kelimeleri
TF-IDF	Sparse weighted term matrix
Sentiment	Pozitif/negatif skor
Embeddings	Dense semantic vector

Modern yaklaşım
Embeddings güçlüdür ama basit length/keyword feature'ları hâlâ tabular modelde ek değer sağlayabilir.

24. Geospatial features

Ham	Feature
lat/lon	distance_to_center
lat/lon	region/geohash
store/customer coords	nearest_store_distance
route data	estimated_travel_time

Coordinate scale
Latitude/longitude'u ham numeric feature olarak kullanmak her model için coğrafi mesafeyi doğru temsil etmez.

25. Feature crosses

İki veya daha fazla kategorinin birleşimi yeni davranış segmenti oluşturabilir.

```
df["device_country"] = (
    df["device"].astype(str) + "__" + df["country"].astype(str)
)
```

Risk
Cross feature cardinality'yi hızla büyütebilir. Rare category handling gerekir.

26. Leakage türleri

Leakage	Örnek
Target leakage	Churn sonrası cancellation_reason
Future leakage	Prediction tarihinden sonraki işlemler
Validation leakage	Tüm dataset'te scaler/encoding fit
Group leakage	Aynı müşteri train ve valid'de
Target encoding leakage	Validation target ile category mean

Feature engineering'in en önemli kuralı
Feature değerini production'daki prediction anında gerçekten hesaplayabiliyor musun? Cevap hayırsa feature muhtemelen leak'tir.

27. Point-in-time correct feature

Feature store ve production ML'de feature geçmişteki belirli prediction timestamp için o anda bilinen verilerden hesaplanmalıdır.

Yanlış	Doğru
Customer lifetime spend bugünün tüm verisiyle	Prediction date'e kadar cumulative spend
Son 30 gün ama geleceği de kapsıyor	Prediction time'dan geriye 30 gün
Current status snapshot	Historical status as-of timestamp

28. Feature selection ile farkı

Feature Engineering	Feature Selection
Yeni representation üretir	Mevcut feature'lardan alt küme seçer
Signal açığa çıkarır	Noise/redundancy azaltır
Domain knowledge yoğun olabilir	Statistical/model-based olabilir

Birlikte kullanılır
Önce anlamlı candidate features üret, sonra CV ve selection/regularization ile gereksizleri ele.

29. Feature importance'a kör güvenme

Feature importance, feature engineering kararını destekleyebilir ama tek doğruluk kaynağı değildir.

- Correlated feature'larda importance bölünebilir.
- Impurity importance high-cardinality feature'ı kayırabilir.
- Permutation importance distribution shift'e duyarlıdır.
- SHAP model davranışını açıklar, nedenselliği değil.

Doğru soru
Feature model skorunu artırıyor mu, stable mı, leakage-safe mi ve business olarak anlamlı mı?

30. Feature ablation

Bir feature ailesini çıkarıp CV performansındaki değişimi ölçmek, gerçek katkıyı anlamanın güçlü yoludur.

Deney	Yorum
Base + RFM	Davranış feature ailesinin katkısı
Base + time windows	Temporal signal katkısı
Base + interactions	Cross-feature katkısı
Base + text	Text representation katkısı

Ablation > importance
Özellikle feature ailesi seviyesinde ablation daha doğrudan deneysel kanıt sağlar.

31. Feature stability

Production'da feature distribution zamanla değişebilir. Stable feature daha güvenilir model davranışı sağlar.

- PSI / distribution shift.
- Missing rate değişimi.
- Category cardinality ve unseen rate.
- Mean/std/quantile drift.

Operational quality
Feature iyi tahmin üretse bile üretimi geç, pahalı veya kararsızsa production için kötü feature olabilir.

32. Feature engineering ve model türü

Model	Feature engineering ihtiyacı
Linear/Logistic	Scaling, encoding, interaction ve nonlinear transforms önemli.
KNN/SVM	Scaling kritik.
Tree/Random Forest	Scaling çoğunlukla gereksiz; categorical handling önemli.
Gradient Boosting	Raw numeric güçlü; domain aggregates çok değerlidir.
Neural Networks	Scaling/normalization ve embeddings yaygın.
Time series model	Lag/rolling/seasonal feature kritik olabilir.

33. Pipeline içinde feature engineering

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), numeric_cols),

    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_cols)
])
```

Reproducibility
Training'de yapılan dönüşüm production inference'ta aynı code path ile çalışmalı.

34. Custom transformer

```
from sklearn.base import BaseEstimator, TransformerMixin

class RatioFeatures(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X["avg_order_value"] = (
            X["revenue"] / X["orders"].clip(lower=1)
        )
        return X
```

Avantaj
Feature logic notebook hücresinde kaybolmaz; test edilebilir ve pipeline'a eklenebilir.

35. SQL ile feature engineering

```
SELECT
    customer_id,
    COUNT(*) AS orders_90d,
    SUM(amount) AS spend_90d,
    AVG(amount) AS avg_order_value_90d,
    MAX(order_date) AS last_order_date
FROM orders
WHERE order_date >= :prediction_date - INTERVAL '90 day'
  AND order_date < :prediction_date
GROUP BY customer_id;
```

As-of semantics
WHERE koşulu prediction_date'i dışarıda bırakıyor; geleceğin feature'a sızmasını engelliyor.

36. Gerçek vaka: churn feature engineering

Feature ailesi	Örnek
Recency	days_since_last_login
Frequency	sessions_7d, sessions_30d
Trend	sessions_7d / sessions_prev_7d
Support	tickets_30d, unresolved_ticket
Payment	failed_payment_90d
Tenure	days_since_signup
Plan	plan_type, monthly_fee
Engagement	feature_usage_diversity_30d

Güçlü sinyal
Absolute usage kadar usage düşüş trendi churn prediction'da daha güçlü olabilir.

37. Gerçek vaka: fraud feature engineering

Feature	Sinyal
txn_count_10min	Velocity
amount_vs_user_median	Kişisel anomali
new_device_flag	Cihaz riski
distance_from_last_txn	Coğrafi anomali
merchant_fraud_rate_oof	Merchant risk - leakage-safe
card_age_days	Kart tenure

Streaming ihtiyacı
Fraud feature'ları real-time hesaplanacaksa offline feature ile online feature parity kritik olur.

38. Gerçek vaka: e-commerce recommendation / propensity

- Category affinity: son 90 günde kategori harcama payı.
- Price sensitivity: indirimli ürün satın alma oranı.
- Recency: son kategori görüntülemeden geçen süre.
- Diversity: benzersiz kategori/brand sayısı.
- Campaign response: geçmiş kampanya conversion rate.

Prediction-time
Kampanya sonucunu tahmin ederken kampanya sonrası davranış hiçbir feature'a giremez.

39. Feature engineering checklist

- Target ve prediction timestamp net mi?
- Feature yalnız geçmiş/erişilebilir bilgiden mi üretiliyor?
- Feature grain target grain ile uyumlu mu?
- Join cardinality kontrol edildi mi?
- Missing/unseen category davranışı tanımlı mı?
- Scaling/encoding train fold içinde mi fit ediliyor?
- Feature business anlamı taşıyor mu?
- Feature cost/latency production için kabul edilebilir mi?
- CV ile gerçek katkısı ölçüldü mü?
- Zaman içinde stability/drift izlenebilir mi?

40. Sık yapılan hatalar

- Rastgele binlerce interaction üretmek.
- Target encoding'i tüm dataset üzerinde yapmak.
- Future rolling window kullanmak.
- Scaler'ı split öncesi fit etmek.
- High-cardinality one-hot ile memory patlatmak.
- Feature importance'ı nedensellik sanmak.
- Train/production feature hesaplama kodlarını ayırmak.
- Ablation yapmadan feature katkısını varsaymak.
- Feature table grain'ini doğrulamamak.
- Timestamp ve timezone mantığını göz ardı etmek.

41. Mülakat soruları

Soru	Kısa cevap
Feature engineering nedir?	Ham veriyi model için daha anlamlı predictive representation'a dönüştürme sürecidir.
Scaling ne zaman kritik?	Distance-based, linear regularized, PCA ve neural yöntemlerde.
Tree model scaling ister mi?	Genellikle hayır; split sıralamasına dayalıdır.
One-hot'ın sorunu?	High-cardinality'de feature explosion.
Target encoding'in en büyük riski?	Target leakage; OOF/fold-aware uygulanmalıdır.
Interaction ne zaman?	Bir feature'ın etkisi diğer feature seviyesine bağlıysa.
Lag feature'da temel risk?	Future leakage.
Rolling mean neden shift edilir?	Current/future target'ı feature içine almamak için.
Cyclical encoding ne için?	Saat/gün/ay gibi döngüsel değişkenler için.
Ablation nedir?	Feature/feature ailesini çıkarıp performans farkını ölçmek.
Feature importance neden yanıltıcı?	Correlation ve model-specific bias nedeniyle.
Point-in-time correctness nedir?	Feature'ın historical prediction anında yalnız o ana kadar bilinen bilgiyle hesaplanması.
Frequency encoding nedir?	Kategori görülme sıklığını sayısal feature yapmak.
RFM nedir?	Recency, Frequency, Monetary.
Feature drift nedir?	Production feature distribution'ın training'e göre değişmesi.

42. Uygulama alıştırmaları

1. Sağ çarpık bir monetary feature için raw, log1p ve RobustScaler sonuçlarını karşılaştır.
2. High-cardinality categorical için one-hot ve frequency encoding ile model performansını kıyasla.
3. OOF target encoding uygula ve naive target encoding ile validation farkını gözlemle.
4. Churn datasında 7d/30d/90d usage feature'ları üret.
5. Lag ve rolling feature'larda deliberate leakage yaratıp skor farkını göster.
6. Cyclical hour encoding ile raw hour feature'ını karşılaştır.
7. Feature interaction ekleyip linear model CV skorundaki değişimi ölç.
8. Feature family ablation tablosu hazırla.
9. SQL ile point-in-time customer features üret.
10. Feature drift raporu için train vs production quantile/missing/category karşılaştırması yap.

43. Mini portföy projesi

Proje: Churn Feature Store - Point-in-Time Correct Feature Engineering

Çıktı	İçerik
01_feature_spec.md	Feature name, source, lookback, grain, freshness.
02_sql_features.sql	7d/30d/90d aggregate ve recency feature'ları.
03_encoding.ipynb	Categorical encoding benchmark.
04_ablation.ipynb	Feature family contribution.
05_leakage_tests.ipynb	Future/target leakage testleri.
src/features.py	Reusable transformations.
report.md	CV gain, stability ve production notes.

CV cümlesi

Built a point-in-time-correct churn feature pipeline with multi-window behavioral aggregates, leakage-safe categorical encoding, feature-family ablation and production-oriented feature specifications.

44. Final cheat sheet

Kavram	Hatırlanacak fikir
Scaling	Model geometry'sini dengeler.
Log transform	Heavy-tail numeric feature.
One-hot	Nominal low/medium cardinality.
Ordinal	Gerçek sırası olan category.
Frequency encoding	Category prevalence.
Target encoding	Güçlü ama leakage riski yüksek.
Interaction	Feature effects depend on each other.
RFM	Recency-Frequency-Monetary.
Lag	Geçmiş değer.
Rolling	Geçmiş pencere özeti.
Cyclical	Saat/gün gibi döngüsel değişken.
Ablation	Gerçek feature katkısını test et.
Point-in-time	Prediction anındaki gerçek bilgi.
Feature drift	Production distribution değişimi.
Pipeline	Train-serving tutarlılığı.

Bu modülden sonra

ML Pipelines bölümünde preprocessing ve feature engineering adımlarını tek reproducible yapı içinde birleştirerek leakage riskini sistematik biçimde azaltabileceksin.